Videometamaterials

하드웨어 환경

- a 2.2 GHz Intel Xeon E5-2650 processor with 256 GB of DDR3 memory at 2.5 GHz. 3Tasks were performed on eight Nvidia Quadro RTX 6000 with 24 GB GDDR6 memory using CUDA 11.7.

Task	Software	Hardware	Runtime
Single design generation and evaluation	Abaqus	CPU (single core)[2]	2.5 minutes[1]
Dataset generation	Abaqus	CPU (16 cores)[2]	8.1 days[1]
Model training	PyTorch in Python	GPU[3]	72 hours
Sample generation	PyTorch in Python	GPU[3]	7.2 seconds per sample (averaged over 16 samples)

- Hyperparameters :

Table S4: Training hyperparameters.

Hyperparameter	Value
Training data size	53,007
Validation data size	4,924
Batch size	32
Learning rate	10^{-4}
Optimization algorithm	Adam[14]
Iterations	200,000
Exponential Moving Average (EMA) decay	0.995
EMA start (iteration)	2,000
EMA update period (iteration)	10
Diffusion timesteps	256
Condition dropout (for classifier-free guidance[8])	0.1
Dropout	none

Abaqus Script 분석

물성

- Flexible 80A

Material properties	TMPTA	Formlabs flexible
Density ρ_s (kg m^{-3})	1100	1100
Young's modulus E_s (MPa)	392.6	8.925
Yield strength σ_{ys} (MPa)	9.03	3.207
Strength at break σ_{fs} (MPa)	14.45	3.207
Strain at break ε_{fs} (-)	0.044	0.424
Poisson's ratio ν_s (-)	0.3	0.48

- a thermoplastic acetal homopolymer resin(Derlin)

Property	**Value**
Elastic Modulus (E)	2.306×10^3 MPa
Poisson's Ratio (ν)	0.35
Damping Coefficient (β)	0.005
Plasticity	MPa, mm/mm
True Stress, True Plastic Strain	40.62, 0.0
	45.24, 0.001133
	52.62, 0.004183
	58.00, 0.0080645
	61.87, 0.012557
	65.81, 0.020035
	69.19, 0.030689
	71.06, 0.038873
	72.61, 0.047114
	73.54, 0.052610
	74.82, 0.06083
	76.74, 0.074477
	78.46, 0.08799
	81.58, 0.11457
	83.00, 0.1276

- 코드 : abaqus_script.py in line 330

```python
# create constitutive model
if material == 'Jin_et_al':
    m.Material(name=material_name)
    E = 2.306e3
    nu = 0.35
    m.materials[material_name].Elastic(table=((E, nu), ))

    m.materials[material_name].Density(table=((density, ), ))
    m.materials[material_name].Damping(beta=0.005)

    m.materials[material_name].Plastic(table=( (40.62, 0.0),
                                    (45.24, 0.001133),
                                    (52.62, 0.004183),
                                    (58.00, 0.0080645),
                                    (61.87, 0.012557),
                                    (65.81, 0.020035),
                                    (69.19, 0.030689),
                                    (71.06, 0.038873),
                                    (72.61, 0.047114),
                                    (73.54, 0.052610),
                                    (74.82, 0.06083),
                                    (76.74, 0.074477),
                                    (78.46, 0.08799),
                                    (81.58, 0.11457),
                                    (83.00, 0.1276)
                                    ))
else:
    m.Material(name=material_name)
    m.materials[material_name].Density(table=((density,), ))
    m.materials[material_name].Hyperelastic(materialType=
        ISOTROPIC, table=((0.192311, 0.288461), ), testData=OFF, type=NEO_HOOKE,
        volumetricResponse=VOLUMETRIC_DATA)
```

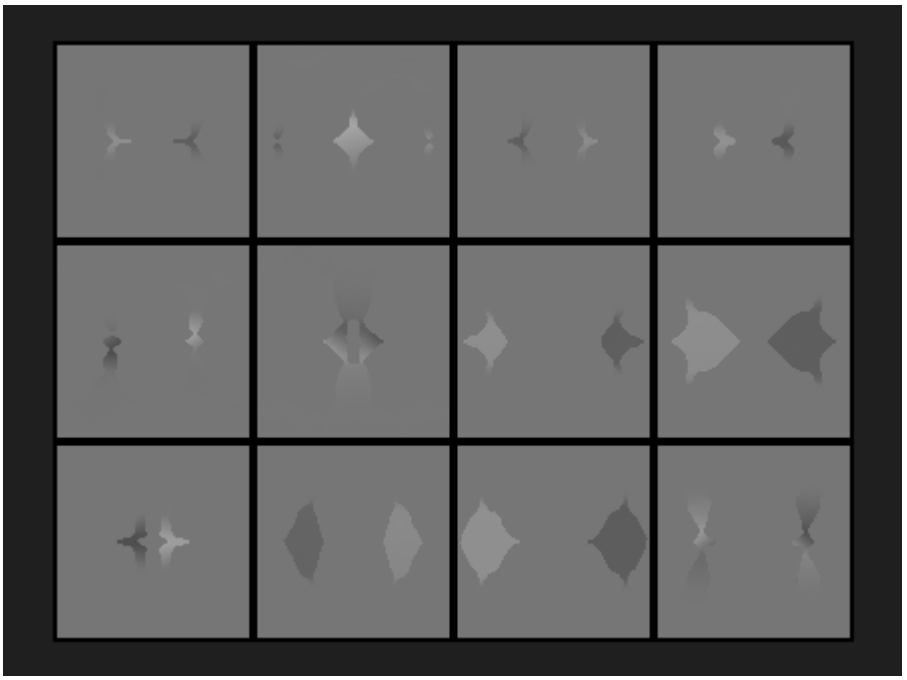
코드 사용 방법

1. VideoMetamaterials > data > target_responses.csv > data

```csv
3.00E+00,3.10E+00,3.40E+00,3.90E+00,4.60E+00,5.50E+00,6.60E+00,7.90E+00,9.40E+00,1.11E+01,1.30E+01
3.00E+00,3.05E+00,3.20E+00,3.45E+00,3.80E+00,4.25E+00,4.80E+00,5.45E+00,6.20E+00,7.05E+00,8.00E+00
3.00E+00,3.03E+00,3.10E+00,3.23E+00,3.40E+00,3.63E+00,3.90E+00,4.23E+00,4.60E+00,5.03E+00,5.50E+00
```

target_response.csv에 출력하고 싶은 물질의 stress-strain커브를 작성한다.
작성 시 0~0.2(20%)dml strain구간에서 0.02단위로 쪼갠 11개 구간에 맞춰서 해당 stress를 작성하면 된다.

2. main.py를 실행한다. main.py의 30번째 줄의 num_preds 변수에 따라 target_response에서 각 줄의 stress-strain커브를 보이는 구조를 생성해준다.

3. 
main 함수 실행 시 해당 폴더 runs/pretrained/eval_target_w_5.0_0에 위와 같은 prediction_channel.gif가 3개 생성된다. 각 prediction channel은 lagrangian frame에서 0은 u_1(x축 방향의 변위), 1은 Von mises 응력 2는 normal stress를 의미한다.
4. 추가적으로 생성된 파일 geometries는 예측한 모든 구조의 개수만큼(위 사진의 경우 12개) geometry가 생성되며 이는 eval_abaqus에서 사용된다.
5. eval_abaqus.py에서 sample_index 변수를 조절해 앞서 뽑은 물질들을 abaqus물질로 돌려볼 수 있다.
이때 sample_grf는 새로 랜덤한 geometries를 생성하여 abaqus를 돌리는 걸로 데이터 셋을 생성하는데 사용한다. create_gifs 역시 FEM을 돌린 구조를 gif로 출력하며 이 역시 데이터셋 출력에 사용된다.